

DeepOHeat su un chip a due materiali

Diario degli esperimenti, parte 2: dal "blob rosso" del primo tentativo a 0,2 °C di errore. 29 agosto 2026.

Il problema in due parole

Stesso chip della parte 1 (1×1×0,5, riscaldatore sulla superficie superiore), ma fatto di **due materiali**: conducibilità $k = 1,4$ per $y < \sim 0,48$ e $k = 0,5$ oltre. Un'interfaccia netta, che nella temperatura produce un *gomito* (la pendenza cambia di un fattore 2,8 attraversandola). La soluzione di riferimento non viene più dal paper ma da **Ansys**, per due casi: il caso 11 (rettangolo di potenza al centro) e il caso 14 (quadrante nell'angolo).

Punto di partenza: il codice dell'autore del PDF (`heat_surface2.py`, `models1.py`, `rescale.py`) estendeva la loss originale con k per regione e un termine d'interfaccia. Risultato: un blob rosso saturo dove dovrebbe esserci un rettangolo tiepido, errore fino a 3 K, e il termine `loss_top` che non scendeva mai. Epoche, batch, peso di `loss_top`, training set misto: niente aiutava. Domanda: *cosa fare?*

Passo 0 — Prima di allenare: la soluzione vera soddisfa la loss?

Controllo che vale sempre la pena fare e che qui ha deciso tutto. Con le differenze finite ho calcolato **ogni termine della loss direttamente sul campo Ansys**. Se la ground truth stessa "viola" la loss, la rete sta imparando un problema diverso da quello di cui abbiamo la soluzione, e nessun trucco di training può convergere.

termine	cosa imponeva il codice	cosa dice il dato Ansys	verdetto
BC top $k \cdot u_z = f$	$f = 1$ sul rettangolo	$k \cdot u_z = 0,20$ (e area un po' più larga)	scala $\times 5$ sbagliata
BC fondo $u - 0,2 = b \cdot k \cdot u_z$	$b = 0,2$	$b = 2,0$ (bilancio energetico, entrambi i casi)	coefficiente $\times 10$ sbagliato
interfaccia	$y = 0,45$ (loss) / 0,5 (modello ϕ)	$y \approx 0,48$; salto di $u_y = 2,83 \approx k_1/k_2$	posizione sbagliata
Laplaciano nelle regioni	$k \cdot \text{lap}(u) = 0$	RMS 0,01 vs 0,34 di uzz	ok

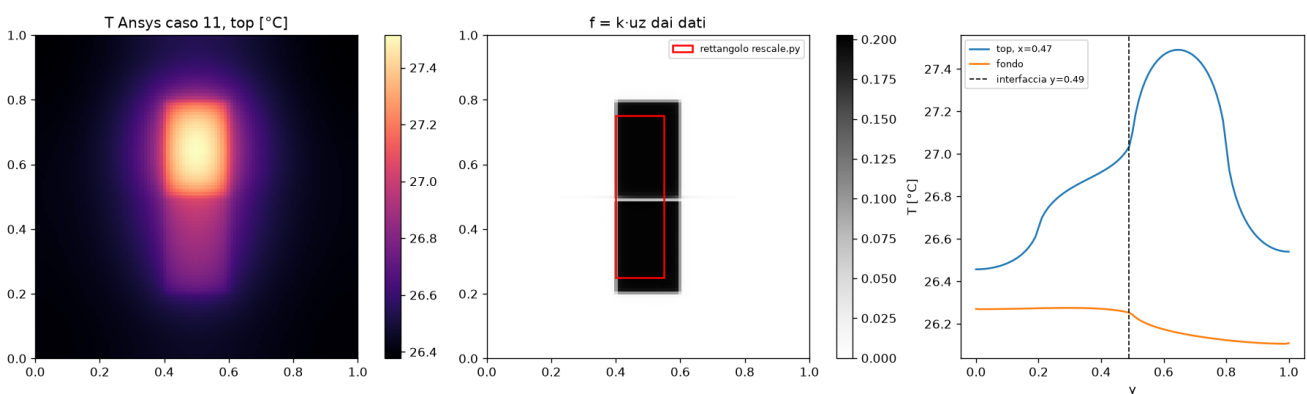


Fig. 1 — Caso 11. Sinistra: temperatura Ansys in superficie. Centro: la potenza *ricavata dai dati* ($k \cdot u_z$ al top) con sopra il rettangolo scritto a mano in `rescale.py`: posizione giusta, ma il livello è 0,2, non 1. Destra: T lungo y con il gomito all'interfaccia.

Il rettangolo di `rescale.py` era **scritto a mano** (la riga che converte il flusso Ansys era commentata) e la formula del paper "0,00625 mW per tile $\leftrightarrow$ 1" non vale per questo setup Ansys. Correzioni: $f = 0,2 \cdot \text{riscaldatore}$, $b = 2,0$, interfaccia 0,48. Dopo la correzione la ground truth dà residui ~ 0 (top 0,036 contro 0,277, fondo 0,006 contro 0,043). Script: `new_experiment/check_truth.py`.

Passo 1 — Il gomito: una feature per il modello

Il trunk in y è una ChebyKAN, cioè polinomi: liscia per costruzione, non può fare un gomito. Modifica minima (`models_kink.py`): il trunk in y riceve $[y, |y - y_i|]$. La funzione $|y - y_i|$ è continua con derivata discontinua: esattamente la forma della soluzione, e grazie alla struttura separabile basta darla al trunk in y . Il termine d'interfaccia diventa la fisica giusta: **continuità del flusso** $k_1 \cdot u_y(-) = k_2 \cdot u_y(+)$, non più "Laplaciano nullo sull'interfaccia" (che chiede regolarità dove la fisica vuole una singolarità).

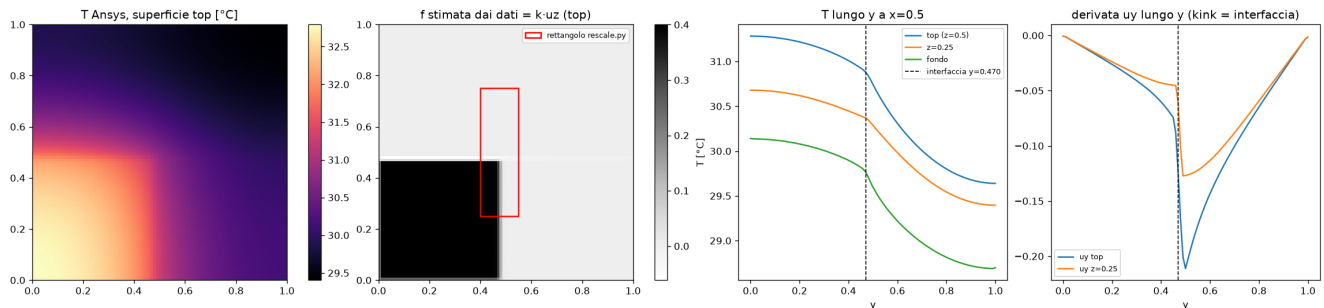


Fig. 2 — Caso 14 (quadrante). Ultimo pannello: la derivata u_y lungo y salta di un fattore $\approx 2,8 = k_1/k_2$ a $y \approx 0,47$: è il gomito che il modello deve saper fare.

Passo 2 — La PINN corretta: 33%. Il modello bara.

Con loss corretta e feature del gomito, il primo run (B0) dà rel L2 = 33%: predizione piatta a 24 °C, **sotto la temperatura ambiente**, fisicamente impossibile con calore in ingresso. Eppure tutti i termini della loss sono $\sim 1e-5$ tranne top. Come si conciliano? Valutando il modello su una griglia fine in z si trova uno **strato limite finto**: nell'ultimo centesimo di spessore uz schizza a $1/k$ (BC top soddisfatta), mentre sotto la temperatura è piatta. La PDE viene controllata solo sui punti di collocazione ($z = 0,05, 0,10, \dots, 0,45$): lo strato sta tra l'ultimo punto e la superficie, dove nessuno guarda. Curvatura $|u_{zz}|$: 260 volte più grande nell'ultimo 0,05 che nel resto.

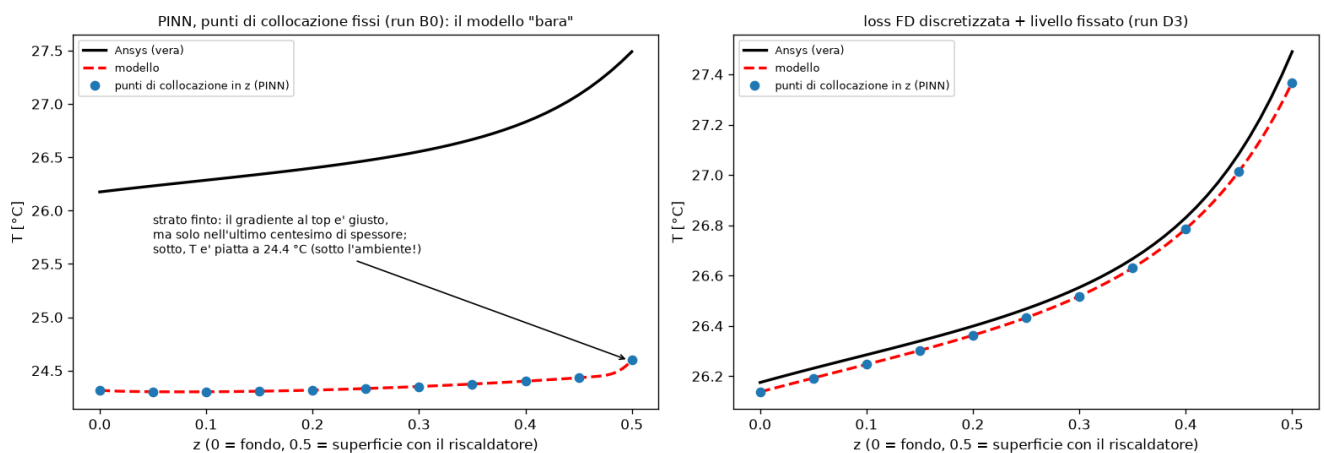


Fig. 3 — Profilo $T(z)$ sotto il riscaldatore, caso 11. Sinistra: il modello PINN (B0) è piatto e sotto l'ambiente, con il gradiente giusto solo nell'ultimo punto. Destra: il modello finale (D3) segue la curva Ansys. I pallini sono i punti di collocazione della PINN.

Rimedio classico: **punti di collocazione casuali** in z a ogni passo (run B1). Lo strato limite sparisce e la **forma** diventa giusta (il bump del rettangolo ha l'ampiezza vera, il gomito c'è), ma l'intero campo è traslato di -2 °C : rel L2 30%.

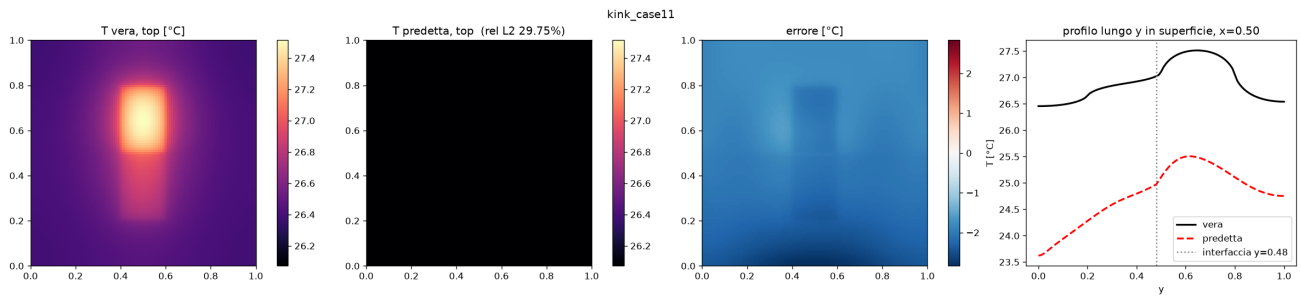


Fig. 4 — Run B1 (PINN, z casuale). Forma corretta, livello sbagliato di 2 °C: la predizione sta ancora sotto l'ambiente.

Diagnosi del livello: misurato su griglia fine, il Laplaciano del modello ha RMS **1,0** (contro 0,03 sui punti di collocazione): oscilla violentemente *tra* i punti, con media positiva. Integrato sul volume è un pozzo di calore: il flusso entra dal top (0,10) e non esce dal fondo (−0,006). Il livello di temperatura è fissato dalla BC di fondo, $u - 0,2 = 2 \cdot k \cdot uz$: un errore di flusso dq nel flusso diventa $dT = 25 \cdot 2 \cdot dq$ nella temperatura. Con $b = 2$ e $f = 0,2$ il livello è **50 volte più sensibile** agli errori di flusso che nel paper ($b = 0,2$, $f \approx 1$). Per questo il codice originale "funzionava" e qui no.

Passo 3 — Loss discretizzata (stile DeepOHeat-v2): 1,3%

Se il problema è "la fisica vale solo dove la si controlla", la risposta è definire la fisica *solo sui punti*: al posto delle derivate con autodiff, uno **stencil alle differenze finite** su una griglia densa (41×41×21), in forma conservativa. Tre vantaggi: non esiste più un "tra i punti" dove barare; l'interfaccia si gestisce con la **media armonica di k** sulle facce, senza termini ad hoc; il bilancio energetico è conservativo per costruzione (l'ho aggiunto anche come termine esplicito). Niente Hessiane → più veloce per punto. È la strada presa da DeepOHeat-v2.

Prima di allenare, di nuovo il Passo 0: lo stencil FD sulla ground truth dà top 7e-4, fondo 3e-5, bilancio 4e-7. Ok.

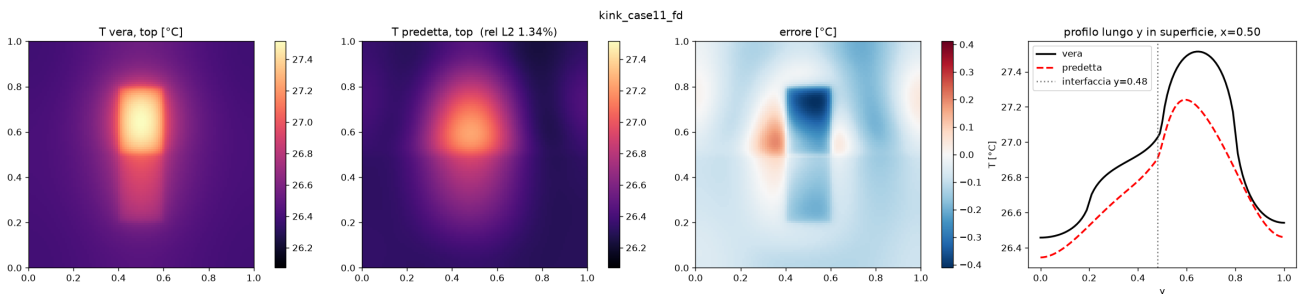


Fig. 5 — Run D0 (loss FD, 30k epoche). Caso 11: rel L2 1,34%, picco −0,27 °C, errore massimo 0,41 °C. Il gomito all'interfaccia (profilo a destra) è riprodotto.

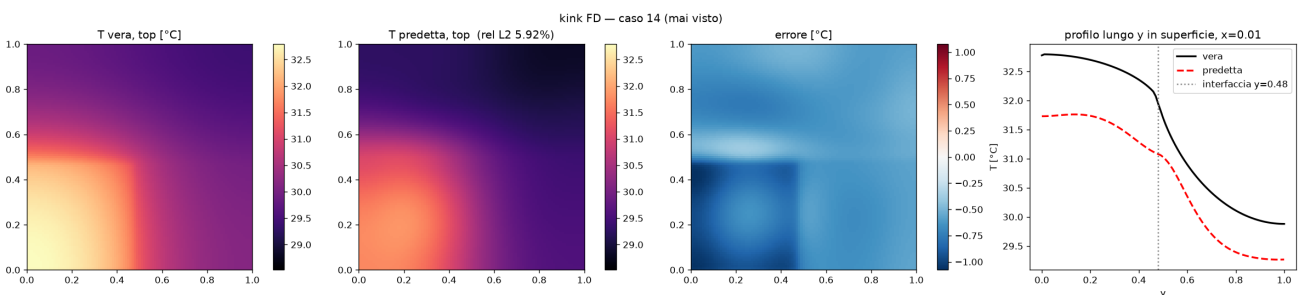


Fig. 6 — Stesso modello D0 sul caso 14, mai usato per costruire nulla: 5,9%. Forma giusta, ma tutto il campo ~0,6 °C sotto.

Passo 4 — Il livello: fisica esatta, e un input troppo debole

In tutti i run FD l'errore di *forma* (tolto l'offset medio) è 0,3–0,9%; quello che domina è un **offset uniforme** di 0,1–0,6 °C, diverso da run a run. Ma il livello si può fissare senza chiederlo alla rete: per Laplace con lati

adiabatici, la temperatura media del fondo è **esattamente** $a + b \cdot \langle f \rangle$ — dipende solo dall'input. Con `--pin_level` aggiungo a ogni mappa la costante che impone questa media: una costante non tocca né la PDE né i flussi, solo lo scalare che la rete sbagliava.

Il vincolo ha funzionato al punto da smascherare l'ultimo colpevole: l'offset residuo era **deterministico** e pari a $25 \cdot b \cdot (\langle f_{\text{input}} \rangle - \langle f_{\text{vero}} \rangle)$. Cioè: il rettangolo scritto a mano ($44 \text{ celle} \times 0,2$) immette il **16% di calore in meno** del riscaldatore Ansys, e anche la mappa ricavata dai dati ne perde un 8% (le celle di bordo, a mezza area, pesate come intere). Con mappe scalate al flusso totale vero (`fs_test_*_flux.npy`):

run D3 (FD + kink + livello fissato), input a flusso esatto	rel L2	offset	picco	max errore	forma
caso 11 — rettangolo centrale	0,72%	−0,04 °C	−0,14 °C	0,22 °C	0,30%
caso 14 — quadrante (mai visto)	1,33%	−0,13 °C	−0,28 °C	0,47 °C	0,41%

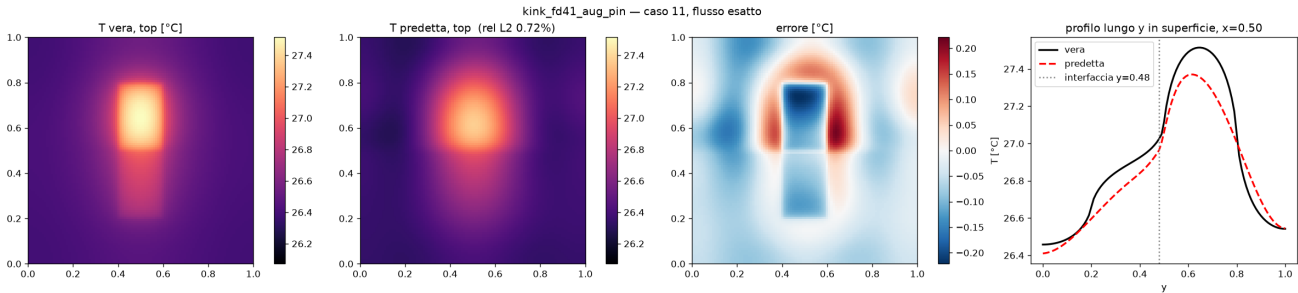


Fig. 7 — Il risultato migliore: run D3, caso 11, input a flusso esatto. Scala dell'errore $\pm 0,2 \text{ °C}$ (nella Fig. 4 era $\pm 2,5 \text{ °C}$). Resta una lieve sottostima ai bordi del riscaldatore: l'input 21×21 non può rappresentare bordi che cadono tra i sensori.

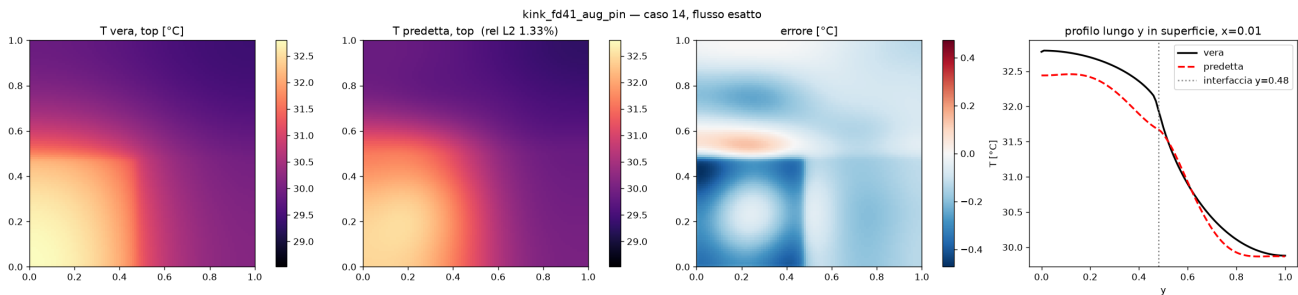


Fig. 8 — Run D3 sul caso 14 (mai visto): 1,33%, errore massimo $0,47 \text{ °C}$ su un campo che varia di $4,3 \text{ °C}$.

Tutti i run in una tabella

cartella	loss	modifiche	caso 11	caso 14
(PDF di partenza)	PINN, k per regione	termine $k_{\text{mean}} \cdot \text{lap}(u) = 0$, $f=1$, $b=0,2$	~3 K di errore	—
B0_pinn_zfixed_CHEAT	PINN	loss corretta + kink	33%	—
B1_pinn_zrandom	PINN	+ z casuale	30%	—
C_pinn_v1_nokink	PINN	come B1 senza $ y-y_i $	24% (interfaccia $300 \times$ peggio)	—
D0_fd	FD $41 \times 41 \times 21$	+ bilancio energetico	1,34% (0,47% input da dati)	5,9%
D1_fd_aug	FD	+ augmentation ampiezza, 50k ep	3,0%	3,0%
D3_fd_aug_pin	FD	+ livello fissato (pin_level)	0,72% (input flusso esatto)	1,33%

Le percentuali sono rel L2 sull'intero volume ($101 \times 101 \times 51$), che include la temperatura ambiente nel denominatore: un offset uniforme di $0,1 \text{ °C}$ vale già ~1,3%. Per l'uso pratico guardare picco e massimo errore in $^{\circ}\text{C}$.

Cosa abbiamo imparato

- Prima di ogni training, verificare che la ground truth soddisfi la loss (`check_truth.py`). Qui tre parametri su quattro erano sbagliati.
- Una PINN a collocazione può soddisfare tutte le BC e ignorare la PDE *tra* i punti. Con griglia fissa fa strati limite finti; con punti casuali fa oscillare il Laplaciano. Il rilevatore "boundary-layer check" in `eval.txt` lo segnala.
- La sensibilità del livello agli errori di flusso è $25 \cdot b$: con $b = 2$ il problema è $10\times$ più severo che nel paper, e il segnale ($f = 0,2$) $5\times$ più piccolo. Stesso codice, problema $50\times$ più difficile.
- La loss discretizzata FD risolve alla radice: fisica definita solo sui punti, interfaccia con media armonica, conservazione dell'energia. $33\% \rightarrow 1\%$.
- Il livello di temperatura è uno scalare noto dalla fisica ($a + b \cdot \langle f \rangle$): fissarlo è gratis e toglie il termine d'errore dominante.
- L'input conta quanto il modello: il rettangolo a mano immetteva il 16% di calore in meno. Con input fedele, D0 passa da 1,34% a 0,47% senza riallenare.
- La feature $|y - y_i|$ è necessaria per la continuità del flusso all'interfaccia ($300\times$ sul termine dedicato).
- Augmentation di ampiezza e griglia FD più fine: effetto nel rumore. Il run D2 (griglia 51) è stato interrotto.

Cosa NON dimostrano questi esperimenti

- $f = 0,2$, $b = 2,0$, $T_{\text{amb}} = 25^\circ\text{C}$ sono **misurati dai dati Ansys**, non presi dal setup: averli dal modello Ansys (q , h , dimensioni) chiuderebbe l'offset residuo del caso 14.
- Due soli casi di test. Un solutore FD nostro (lo stencil c'è già; pyamg è installato) darebbe test set illimitati e permetterebbe di misurare la qualità del riferimento Ansys stesso, che è stato generato su $21 \times 21 \times 11$ e poi interpolato.
- Un solo seed per configurazione: differenze sotto $\sim 0,3\%$ di rel L2 sono rumore.
- Sotto $0,2\text{--}0,5^\circ\text{C}$ di errore stiamo probabilmente fittando gli artefatti del riferimento: spingere ancora l'accuratezza su questi due casi non è un buon investimento. Le direzioni con margine sono un solutore per i test, k come input del modello (un modello per molte configurazioni) e geometrie a più strati.

Appendice — Riprodurre

Dalla root della repo, venv attivo. Training set: lo stesso misto GRF + blocchi della parte 1.

```
# verifica della loss sulla ground truth
python new_experiment/check_truth.py --u data/u_test_11.npy --f data/fs_test_11_flux.npy
# modello finale (D3), valutato sui due casi
python new_experiment/heat_surface3.py --model kink --loss fd --fd_nx 41 --fd_nz 21
--lam_energy 1 \
  --amp_aug_min 0.1 --pin_level 1 --epochs 30000 --decay_steps 600 --tag _fd41_aug_pin \
  --test_f data/fs_test_11_flux.npy,data/fs_test_14_flux.npy --test_u
data/u_test_11.npy,data/u_test_14.npy
# controparte PINN che fallisce (B1): --loss pinn --z_random 1
```

File: `new_experiment/heat_surface3.py` (loss PINN/FD, BC parametriche, `pin_level`, augmentation, eval multi-caso), `new_experiment/models_kink.py`, `new_experiment/check_truth.py`. I file di partenza dell'autore sono conservati intatti nella stessa cartella. Risultati e pesi in `results/two_materials/`.